

Hapi's Havoc – Unity Prototype

1 High-level game concept

- A domino-shaped, 6 × 6 river board.
 - Players roll 3 dice each turn (2 × blue = path tiles, 1 × red = obstacle tile), then **insert** those tiles from the chosen board edge; the whole row shifts and the far tile drops back into the bag. For now we use arrows on each side of the rows to do it manually.
 - Each player controls a **boat** that
 - spawns at its home bank (bottom edge),
 - spends **movement points** to hop between snap-points, following the curved paths drawn on the tiles,
 - may reverse along the path or return to its bank, and
 - later delivers cargo / scores (mechanics still to be added).
-

2 Current Unity architecture

Area	Main script(s)	Status
Tile definition	<code>TileLibrary</code> (ScriptableObject) → holds the 13 tile templates listed below	✓ done
Tile instancing	<code>TileInstance</code> (+ <code>PathVisualizer</code>)	✓ tiles spawn, paths render (straight, quarter-arc, half-arc, S-curve)
Bag logic	<code>TileBagManager</code>	✓ basic draw / return works

Board grid	GridManager	✓ initial 6×6 board spawns ▪ arrows around board let you push rows manually ▪ row-push instant (no tween yet)
Arrow UI (debug)	RowArrow + helpers in GridManager	✓ world-space quads for now
Boat control	BoatController (large, WIP)	• bank → first row works (left/right snap) • river-to-river works forward • reverse & U-path logic implemented but still buggy • bank return option added
Editor gizmos	DrawSnapGizmos	✓ snap-point visual aid
Visual polish	URP Unlit material on paths; basic lift/hover tweens on boat & tiles	basic

3 Tile catalogue (domino tiles are horizontal, 6 snap-points)

Snap-point indices

0: top-left 1: top-right 2: bottom-left 3: bottom-right 4: right edge 5: left edge

ID	Name	Path connections
1	TileSimple 01	0-5, 2-1, 3-4
2	TileSimple 02	2-5, 0-3, 1-4
3	TileFace (<i>U-shape</i>)	0-5, 1-4, 2-3
4	TileCross	0-2, 1-3, 5-4
5	TileTurn 01	0-5, 2-4, 1-3
6	TileTurn 02	0-2, 1-4, 3-5
7	TileTurn 03	0-5, 2-5, 2-1, 3-4
8	TileTurn 04	0-5, 1-4, 2-5, 0-3

9	TileChange	0-5, 2-5, 1-4, 3-4
10	TileTurnChange 01	0-5, 2-5, 2-4, 1-3
11	TileTurnChange 02	3-5, 0-2, 1-4, 3-4
12	TileThroughTurn 01	0-5, 0-2, 1-3, 3-4
13	TileThroughTurn 02	2-5, 0-2, 1-3, 1-4
–	<i>(Optional max tile – disabled)</i>	0-5, 2-5, 0-3, 1-4, 3-4, 1-2

All paths are bidirectional; tiles may spawn flipped (obstacle side up) or rotated 180°.

Left-side snaps = {0, 2, 5} Right-side snaps = {1, 3, 4} - when not rotated

4 What already works

Current Stable & Working Features

The project has a robust and stable foundation for the core gameplay mechanics.

- **Grid & Tile System:**
 - A 6x6 grid is procedurally generated at runtime.
 - **Seeding:** The grid generation can be controlled by a specific integer seed for reproducible layouts, or randomized each time.
 - **Row Pushing:** Rows can be pushed smoothly from either side, with new tiles sliding in and old tiles being ejected. The falling tile animation is non-blocking to ensure fluid gameplay.
 - **Tile Bag:** TileBagManager correctly handles drawing tiles from a finite bag and returning ejected tiles.
- **Core Boat Movement & Selection:**
 - **Selection/Deselection:** The boat can be selected (lifting and bobbing) and deselected.
 - **Path Navigation:** The boat correctly moves along all defined path types on blue tiles (straights, curves, U-turns).
 - **Reverse Movement:** The boat can correctly reverse its last move.
- **Bank Interaction:**
 - The boat can successfully move from a bank to a valid tile in an adjacent row.
 - The boat can successfully move from a tile on an edge row back to the corresponding bank, including along reverse paths and U-turns that lead off the board.
- **Reversed Tile Interaction ("Push-Your-Luck" Foundation):**
 - The pathfinding logic correctly "sees through" adjacent reversed (red) tiles, allowing the boat to find the next valid blue tile in a column.

- This works both when embarking from a bank and when moving from tile to tile.
- Movement across reversed tiles is correctly forced to be straight-line.
- The BoatController successfully tracks the path of reversed tiles crossed in a single move, ready for penalty application once cargo mechanics are implemented.
- **Turn Structure & Gameplay Loop (Partial Implementation):**
 - **Movement Points Enforced:** The boat is limited by maxMovementPoints (currently 3). It cannot start a new move if its points are zero.
 - **Chained Moves:** The boat remains selected after a move, allowing the player to chain multiple moves within a single turn until points run out.
 - **Automatic Turn End:** When the boat makes its last move, it automatically deselects and settles down, providing clear visual feedback that the movement phase is over.
 - **Turn Reset:** A new turn can be started by clicking the boat again, which correctly resets its movement points.
- **Ejection & Animation System:**
 - **Ejection Survival:** The boat correctly survives being on a tile that is pushed off the board.
 - **Placement Logic:** The boat correctly identifies its new landing spot by searching past any reversed tiles in the destination column, or by landing on the bank if the column is fully blocked.
 - **Polished Animations:** The entire ejection sequence is animated. The boat fades out as the tiles slide, waits for a short delay, then fades back in while smoothly settling onto its new position (either a tile or a bank).
 - **Visual Fidelity:** A custom Lit shader graph is used for the boat and tiles, allowing for dynamic transparency (fading) while maintaining correct depth sorting and full reactions to scene lighting, preventing visual glitches. Shadows are also correctly disabled during fade animations.

What We Should Work On Next

You are right, the next logical step is to build a formal **Game/Turn Manager**.

Currently, the "turn" is managed entirely inside the BoatController. This is fine for a single boat, but it's not scalable and doesn't account for the other phases of a turn described in your rulebook (like "Storm" or "Assign Workers").

The next big step is to create a high-level "conductor" script that manages the flow of the game. Let's call it **GameManager.cs**.

This new manager would be responsible for:

1. **Tracking Game State:** It would have an enum for the current game phase (e.g., **Player1_Movement**, **Player2_Movement**, **StormPhase**, etc.).

2. **Managing Turns:** It would know whose turn it is. At the end of one player's movement, this manager would be responsible for starting the next player's turn or the next game phase.
3. **Controlling Player Input:** It could enable/disable the BoatController scripts so that only the current player can move their boat.
4. **Communicating with Other Managers:** Instead of the player pressing 'R' to reset points, the GameManager would call `boat.ResetMovementPoints()` at the official start of that boat's turn.